MSc in Mathematical Engineering
Course: Advanced Programming for Scientific Computing

Lecturer: Luca Formaggia
Teaching Assistant: Paolo Joseph Baioni

Course project
A.Y. 2024/2025

FlowPDE: Flow-Based Generative Surrogates for Parametrized PDEs

---

Authors:

Sarper Yurtseven

Mentors:

Andrea Manzoni
Nicola Farenga
Giacomo Bottacini

Abstract

This report presents `flowpde`, a PyTorch library that models the conditional distribution $p(\boldsymbol{u} \mid \boldsymbol{\mu})$ with continuous-time flows. It fits the same conditional neural ODE either by exact maximum likelihood or by flow matching, and its modular design keeps the dynamics, the training objective, the probability path, the time sampler, the coupling and the source distribution as separate components.

Thirteen controlled experiments on Poisson, Burgers and Darcy problems vary conditioning, backbone, training objective, flow-matching components and observation quality, one axis at a time. Removing the conditioning increases the median error by 17×, while the loss curve still looks healthy. The smallest of four backbones is the most accurate on every forward problem. On frozen weights, Euler sampling beats fourth-order Runge–Kutta by up to 8.6× at equal cost, and integrating the ODE to convergence is 11% worse than stopping at sixteen Euler steps. Maximum-likelihood training improves test likelihood, worsens sample accuracy, and costs 66× more than flow matching.

For the inverse problems, the relative L^2 error of a single draw hides about half of what conditioning buys, whereas proper scoring rules show it. The learned posterior narrows by 3.9× from the flattest to the steepest flux decile, which reproduces the identifiability structure of Darcy flow without being told it. Against a converged reference chain the learned sampler is five to ten times less accurate, but it is cheaper per observation and the two costs break even after ten inversions. The results are limited to one run per cell in most settings, they include no deterministic baseline, and they show no speed advantage over the reference solver at this scale.

Contents

Abstract **1**

1 Introduction **3**
- 1.1 Contributions . . . 3
- 1.2 Related work . . . 4

2 Generative AI for PDE surrogates **4**
- 2.1 Flow-based generative modeling . . . 4
  - 2.1.1 Continuous normalizing flows . . . 5
  - 2.1.2 Flow matching . . . 6
  - 2.1.3 Rectified flows . . . 7
- 2.2 Application to parametrized PDEs . . . 8
  - 2.2.1 Forward problems . . . 8
  - 2.2.2 Inverse problems . . . 9

3 FlowPDE **9**
- 3.1 Library overview . . . 10
- 3.2 Modules and implementation details . . . 11
  - 3.2.1 `flowpde.core` . . . 11
  - 3.2.2 `flowpde.models` . . . 11
  - 3.2.3 `flowpde.flows` . . . 11
  - 3.2.4 `flowpde.objectives` . . . 11
  - 3.2.5 `flowpde.trainers` . . . 12
  - 3.2.6 `flowpde.datasets` . . . 13
  - 3.2.7 `flowpde.solvers` . . . 13
- 3.3 Installation and usage . . . 13

4 Numerical experiments **13**
- 4.1 Setup . . . 14
- 4.2 Conditioning, backbone and objective . . . 16
- 4.3 Accuracy against sampling cost . . . 17
- 4.4 Inverse problems and calibration . . . 19
- 4.5 Controlling how ill-posed the problem is . . . 21
- 4.6 Comparison against MCMC . . . 22
- 4.7 Threats to validity . . . 24

5 Conclusions **24**

Bibliography **25**

1 Introduction

Solving a partial differential equation once is not a hard problem anymore. Solving it ten thousand times still is, and ten thousand times is what the applications actually ask for.

Consider what these applications do. Design optimization sweeps a parameter space. Optimal control differentiates through repeated solves. Uncertainty propagation samples the input distribution. Inverse problems search over inputs until the outputs match a measurement. In every one of them the object we care about is not a single solution but the map

$$\boldsymbol{\mu} \mapsto u(\boldsymbol{\mu}),$$

queried again and again. And here is the difficulty. A fine discretization is what makes each individual solve trustworthy, and it is also what makes ten thousand of them too expensive.

Reduced order modelling is the classical answer to this. Its central idea is an accounting one. We split the work into an expensive offline stage, run once, and a cheap online stage, run per query [26]. Deep learning has since supplied nonlinear replacements for the linear reduced subspace [9], and neural operators that learn the solution map directly as a map between function spaces [18, 19].

These methods work well on the standard benchmarks. They also share one property that is easy to overlook, because almost all of them have it: they are *deterministic*. Each returns one field per input, and each is trained under a mean squared error whose population minimizer is the conditional mean

$$\mathbb{E}[u \mid \boldsymbol{\mu}].$$

Where the solution map is a genuine function this is the right thing to return, since the conditional mean of a point mass is the point itself.

The difficulty is that several of the applications that motivate reduced order modelling most strongly are the ones where the map is *not* a function. Recovering a coefficient field from sparse noisy measurements does not determine that field. Reconstructing a solution from partial observations does not determine the solution. Predicting a chaotic system far enough ahead does not determine the trajectory, even when the statistics stay well determined. In each of these the conditional mean is an average taken over candidates that contradict each other.

Now, why is that bad? Because averaging solutions of a nonlinear equation does not give a solution. It gives a smooth field that satisfies nothing, and the model returns it without any sign that something was lost.

This report takes the other route. We model the conditional distribution

$$p(u \mid \boldsymbol{\mu})$$

and we draw samples from it. Individual samples can then be sharp and consistent with the equation instead of averaged, and the spread across samples estimates what the data left undetermined.

The tool we use is the family of continuous-time flow-based generative models: continuous normalizing flows [3, 11] and, above all, the simulation-free flow matching objectives that have largely replaced them [1, 20, 23]. Two of their properties suit surrogate modelling in particular.

Sampling is an ODE solve. The number of function evaluations is therefore an accuracy-against-cost dial, and it can be turned after training, on fixed weights.

Conditional training is amortized. One training run gives a sampler for the whole family $\{p(\cdot \mid \boldsymbol{\mu})\}_{\boldsymbol{\mu}}$. This is the same offline/online split as in reduced order modelling, reached from the machine-learning side.

What we contribute is a library, `flowpde`, rather than a single algorithm. The reason is that the design space of flow-based PDE surrogates is large. It contains the interpolation path, the time schedule, the noise–data coupling, the source distribution, the backbone, the training objective, the solver and the step count. Published comparisons between these axes are rare, and part of the reason is simple: each variant usually arrives as its own codebase. If the axes are separate, swappable components instead, then comparing them is a configuration change and not a reimplementation.

1.1 Contributions

- **The flow is separated from the objective.** The dynamics are one object and the loss that fits them is another. Hence the same conditional neural ODE can be trained by exact maximum likelihood or by flow matching, with no change to model, data or sampler, and comparing simulation-based against simulation-free training becomes a controlled experiment rather than a comparison between two codebases (Section 3.1).

- **Flow matching is factored into four independent components.** Path, time sampler, coupling and source can each be replaced on their own. Meaning that the variants of Section 2.1.2 are *configurations* of one implementation and not subclasses of it (Section 3.2). Section 4.3 uses this to compare six of them under one protocol.
- **Reflow's correctness condition is enforced by the structure.** Reflow straightens trajectories only if training uses the same source point that generated each target (Remark 4). Drawing fresh noise instead makes it a no-op that the loss curve does not show. Making the source an explicit component turns that condition into something a test can check.
- **Forward and inverse generators with controllable ill-posedness.** Poisson, Burgers and Darcy problems are generated through one spectral pipeline, in either direction, with observation noise and random masking as explicit knobs. Section 4.5 turns those knobs from a posterior that is no better than the prior to one that is genuinely informative.
- **Distributional evaluation, not relative L^2 alone.** Proper scoring rules, coverage, rank histograms, spread–skill ratios and a variance decomposition, all reported in physical units (Section 2.2.2).
- **A controlled study across the design space.** Conditioning mechanism, backbone, training objective, flow-matching components and observation quality are varied on shared data, normalization, optimizer and sampler, so that each comparison isolates one axis. The cheap cells are repeated at matched seeds (Section 4).

1.2 Related work

Flow-based generative modelling. Normalizing flows [27] became continuous with neural ODEs [3], and scalable with free-form dynamics [11]. Diffusion and score-based models [14, 29] then dominated for several years. After that, flow matching [20], rectified flow [23] and stochastic interpolants [1] unified and simplified all of it. The main refinements since are minibatch-OT couplings [32] and time-schedule design [7]. Section 2.1 develops this material, and Lipman et al. [21] is the current reference treatment.

Generative models for PDEs. PDE-Refiner [22] rewrites a solver update as a short denoising process, and by doing so recovers the low-amplitude spectral content that a mean squared error ignores. Kohl et al. [17] benchmark autoregressive conditional diffusion for turbulence, and find its long-rollout statistics much better than those of deterministic predictors. GenCast [25] shows the same approach working at operational scale for weather.

Two features of this literature matter for what follows. It is mostly *diffusion* based rather than flow matching based, and it is mostly about *time-dependent* problems, where the probabilistic argument is easiest to make. Parametrized elliptic and parabolic problems, treated in both directions, with the flow matching objective, and with calibration reported instead of assumed, are covered much less often. That is the gap this work sits in.

The inverse half has one direct predecessor. Wildberger et al. [33] train a conditional flow matching model as an amortized posterior estimator, and that is exactly the construction Section 2.2.2 adopts.

Section 2 reviews flow-based generative modelling and its application to parametrized PDEs. Section 3 describes the library. Section 4 reports the numerical experiments.

2 Generative AI for PDE surrogates

This section sets up the machinery the rest of the report rests on. First, Section 2.1 defines what a flow-based generative model is and gives three ways of training it. Then Section 2.2 connects that to partial differential equations, where we answer two questions: what the distribution is over, and when modelling one is worth the trouble.

2.1 Flow-based generative modeling

We are given an unknown distribution q on $\mathbb{R}^d$, known only through samples, and we want more samples from it. The flow-based idea is to *move* a simple reference $\rho_0 = \mathcal{N}(\mathbf{0}, I)$ onto q.

Let $\boldsymbol{v}_t(\boldsymbol{x})$ be a time-dependent velocity field on $[0,1] \times \mathbb{R}^d$, and let ψ_t be its flow map, defined by

$$\frac{\mathrm{d}}{\mathrm{d}t}\psi_t(\boldsymbol{x}_0) = \boldsymbol{v}_t\big(\psi_t(\boldsymbol{x}_0)\big), \qquad \psi_0(\boldsymbol{x}_0) = \boldsymbol{x}_0. \tag{1}$$

If $\boldsymbol{v}$ is Lipschitz in $\boldsymbol{x}$, then Picard–Lindelöf makes ψ_t a diffeomorphism for every t, and we obtain the inverse simply by integrating backwards. *Invertibility is therefore automatic*, and Section 2.1.1 shows this to be the main advantage of the continuous formulation.

Pushing ρ_0 through the flow gives a curve of measures $p_t = (\psi_t)_{\#}\rho_0$, which we call the *probability path*. Equivalently, it is the solution of the continuity equation

$$\partial_t p_t(\boldsymbol{x}) + \nabla \cdot \big(p_t(\boldsymbol{x})\, \boldsymbol{v}_t(\boldsymbol{x})\big) = 0, \qquad p_0 = \rho_0. \tag{2}$$

We say that $\boldsymbol{v}$ *generates* p_t. The generative task is then just the boundary condition $p_1 = q$. Many fields satisfy it, so the methods below differ not in what they look for, but in the loss they use to find it.

Remark 1 (One object, three training routes)**.** *The three subsections that follow all describe the same thing: a velocity field $\boldsymbol{v}_{\boldsymbol{\theta}}$ and the path it generates. They differ only in the objective. Section 2.1.1 maximizes exact likelihood, which requires an ODE solve inside every gradient step. Section 2.1.2 regresses against a closed-form target, and never solves an ODE while training. Section 2.1.3 keeps that regression but changes which pairs of points get joined, so as to straighten the trajectories. In other words, continuous normalizing flows, flow matching and rectified flow are one model family and three training procedures. In FlowPDE they are literally the same class fitted by different objectives.*

2.1.1 Continuous normalizing flows

A classical normalizing flow [27] stacks simple invertible layers, so that the change-of-variables formula gives us the exact density

$$\log p_1(\boldsymbol{x}) = \log \rho_0\big(\psi^{-1}(\boldsymbol{x})\big) - \sum_{k=1}^{K} \log \left| \det \frac{\partial \psi^{(k)}}{\partial \boldsymbol{x}} \right|. \tag{3}$$

That exactness is what separates normalizing flows from variational autoencoders and adversarial networks. The price we pay for it is the determinant. A general one costs $\mathcal{O}(d^3)$, so the layers have to be triangular or coupling-based. In other words, the architecture ends up being designed around the Jacobian rather than around the problem.

Chen et al. [3] removed this trade-off by taking the depth to infinity. Parametrize $\boldsymbol{v}_{\boldsymbol{\theta}}$ in (1) by any network we like; invertibility now follows from uniqueness of ODE solutions. The log-determinant is replaced by the *instantaneous change of variables*,

$$\frac{\mathrm{d} \log p_t}{\mathrm{d}t} = -\operatorname{tr}\Big(\frac{\partial \boldsymbol{v}_t}{\partial \boldsymbol{x}}\Big)$$

along a trajectory, which follows from (2) by expanding the divergence and dividing by p_t.

So the $\mathcal{O}(d^3)$ determinant has become an $\mathcal{O}(d)$ trace, and the sum over layers has become an integral:

$$\log p_1(\boldsymbol{x}_1) = \log \rho_0(\boldsymbol{x}_0) - \int_0^1 \operatorname{tr}\left(\frac{\partial \boldsymbol{v}_{\boldsymbol{\theta}}}{\partial \boldsymbol{x}}\big(\boldsymbol{x}(t), t\big) \right) \mathrm{d}t. \tag{4}$$

Both terms come out of one backward solve of the augmented system $[\boldsymbol{x}, \log p]$, and training minimizes

$$\mathscr{L}_{\mathrm{NLL}}(\boldsymbol{\theta}) = -\,\mathbb{E}_{\boldsymbol{x}_1 \sim q}\big[\log p_1^{\boldsymbol{\theta}}(\boldsymbol{x}_1)\big]. \tag{5}$$

The trace still costs d Jacobian-vector products. FFJORD [11] removes that too, using the Hutchinson estimator [15]: for $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$, $\operatorname{Cov}(\boldsymbol{\varepsilon}) = I$,

$$\operatorname{tr}(A) = \mathbb{E}_{\boldsymbol{\varepsilon}}\big[\boldsymbol{\varepsilon}^{\top} A\, \boldsymbol{\varepsilon}\big], \qquad \operatorname{tr}\left(\frac{\partial \boldsymbol{v}_{\boldsymbol{\theta}}}{\partial \boldsymbol{x}}\right) \approx \frac{1}{M} \sum_{m=1}^{M} \boldsymbol{\varepsilon}_m^{\top} \left(\frac{\partial \boldsymbol{v}_{\boldsymbol{\theta}}}{\partial \boldsymbol{x}} \boldsymbol{\varepsilon}_m \right), \tag{6}$$

where each term is one backward pass. It is unbiased for any $M \geq 1$, but its variance grows with d, which is why M is sometimes raised.

This leaves three costs, and together they motivate Section 2.1.2.

Training requires simulation. Every gradient step contains tens to hundreds of network evaluations.

The dynamics get worse as training proceeds. Nothing in (5) rewards well-behaved dynamics, because the likelihood depends on $\boldsymbol{v}_{\boldsymbol{\theta}}$ only through the map at $t = 1$. So the field stiffens during training and the cost per step *grows* [11].

The usual fixes are added afterwards. They are kinetic-energy or Jacobian penalties, which tell the model to prefer straight-line transport. Flow matching gets the same thing by construction.

Conditioning changes nothing structural here. We write $\boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{x}, t; \boldsymbol{c})$ and replace q by $q(\cdot \mid \boldsymbol{c})$.

2.1.2 Flow matching

Suppose for a moment that we already knew a path p_t from ρ_0 to q, and a field $\boldsymbol{u}_t$ generating it. Then fitting $\boldsymbol{v}_{\boldsymbol{\theta}}$ would be plain regression, with no ODE solve anywhere [20]. Of course we know neither.

The way out is to build the path as a mixture of elementary paths that we do know in closed form. Condition on $\boldsymbol{x}_1 \sim q$ and pick a conditional path with

$$p_0(\cdot \mid \boldsymbol{x}_1) = \rho_0, \qquad p_1(\cdot \mid \boldsymbol{x}_1) \approx \delta_{\boldsymbol{x}_1}, \tag{7}$$

so that each one carries the whole reference onto a single data point. Marginalizing then gives us exactly the boundary conditions we wanted,

$$p_t(\boldsymbol{x}) = \int p_t(\boldsymbol{x} \mid \boldsymbol{x}_1)\, q(\boldsymbol{x}_1)\, \mathrm{d}\boldsymbol{x}_1, \qquad p_0 = \rho_0, \quad p_1 = q. \tag{8}$$

The conditional field $\boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1)$ is analytic. The marginal one,

$$\boldsymbol{u}_t(\boldsymbol{x}) = \int \boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1) \frac{p_t(\boldsymbol{x} \mid \boldsymbol{x}_1)\, q(\boldsymbol{x}_1)}{p_t(\boldsymbol{x})}\, \mathrm{d}\boldsymbol{x}_1, \tag{9}$$

is an intractable integral against the posterior over which data point produced $\boldsymbol{x}$. As we will see, it never has to be evaluated.

Theorem 1 (Conditional flow matching; 20)**.** *Let p_t and $\boldsymbol{u}_t$ be the marginals* (8) *and* (9)*. Then $\boldsymbol{u}_t$ generates p_t, and with*

$$\mathscr{L}_{\mathrm{CFM}}(\boldsymbol{\theta}) = \mathbb{E}_{t,\, \boldsymbol{x}_1 \sim q,\, \boldsymbol{x} \sim p_t(\cdot \mid \boldsymbol{x}_1)} \left\| \boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{x}, t) - \boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1) \right\|^2, \tag{10}$$

the gradients of the marginal and conditional objectives coincide, so the two have the same minimizers in $\boldsymbol{\theta}$.

Sketch of proof. Expand both squared norms. The $\|\boldsymbol{v}_{\boldsymbol{\theta}}\|^2$ terms agree, because $\boldsymbol{x}$ has the same marginal law under both expectations. The $\boldsymbol{\theta}$-free terms vanish under the gradient. And the cross terms are equal once we substitute (9). □

This means that a target we cannot compute has been replaced by one we can, at no cost in the minimizer. One consequence matters later. The model cannot fit the conditional target exactly, because many pairs give the same $\boldsymbol{x}_t$, so it converges to their conditional average, which is $\boldsymbol{u}_t$. *The residual therefore never reaches zero.* That floor is why Section 3.2.5 selects models on sampled error and not on the loss.

Gaussian paths, and diffusion as a special case. The standard conditional family is Gaussian, $p_t(\boldsymbol{x} \mid \boldsymbol{x}_1) = \mathscr{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_t(\boldsymbol{x}_1), \sigma_t(\boldsymbol{x}_1)^2 I)$, whose generating field is

$$\boldsymbol{u}_t(\boldsymbol{x} \mid \boldsymbol{x}_1) = \frac{\sigma_t'}{\sigma_t}(\boldsymbol{x} - \boldsymbol{\mu}_t) + \boldsymbol{\mu}_t'.$$

Different schedules $(\boldsymbol{\mu}_t, \sigma_t)$ recover different known models. Two of them are the variance-preserving and the variance-exploding schedule of score-based diffusion [14, 29], which places diffusion inside this framework: its probability-flow ODE is (1) for one particular Gaussian path.

Those schedules are not the natural choice, though. They come from a noising process that has to run to $t \to \infty$ to forget the data, and so they are curved. Lipman et al. [20] use the optimal-transport path instead,

$$\boldsymbol{\mu}_t(\boldsymbol{x}_1) = t\, \boldsymbol{x}_1, \qquad \sigma_t = 1 - (1 - \sigma_{\min})t, \tag{11}$$

where mean and standard deviation both move at constant rate. As $\sigma_{\min} \to 0$ this collapses to

$$\boldsymbol{x}_t = (1 - t)\, \boldsymbol{x}_0 + t\, \boldsymbol{x}_1, \qquad \boldsymbol{u}_t(\boldsymbol{x}_t \mid \boldsymbol{x}_0, \boldsymbol{x}_1) = \boldsymbol{x}_1 - \boldsymbol{x}_0 : \tag{12}$$

the straight segment from a noise sample to a data sample, together with its constant velocity. Nothing in the final objective is more complicated than that.

Remark 2 (Interpolation and velocity must agree)**.** *In* (12) *the target velocity is the* exact time derivative *of the interpolant. This is a requirement and not a convention. If the two disagree, then the model sees points from one path and regresses the velocity of another, and it converges to something that generates neither. Any implementation that offers several paths must treat $\boldsymbol{u}_t = \partial_t \boldsymbol{x}_t$ as a tested invariant, and that is how FlowPDE handles it.*

Couplings. Theorem 1 assumes the independent coupling $\pi = \rho_0 \otimes q$. But it holds for any π with the right marginals [1, 32], and the choice turns out to matter. Under independence the target at a given $(\boldsymbol{x}_t, t)$ varies a lot across draws, and that variance is what sets the noise floor.

Minibatch optimal transport [32] reduces it by pairing within a batch. We solve

$$\sigma^\star = \arg\min_{\sigma \in S_B} \sum_{i=1}^{B} \left\| \boldsymbol{x}_0^{(\sigma(i))} - \boldsymbol{x}_1^{(i)} \right\|^2 \tag{13}$$

and train on the reordered pairs, so the segments get shorter and the targets agree more closely. Two caveats come with it. The minibatch plan is a *biased* estimator of the population plan, with bias falling as B grows, and solving (13) exactly costs $\mathcal{O}(B^3)$.

The stochastic-interpolant view [1] starts from the interpolant rather than the density, and it makes two further axes explicit. The *source* $\boldsymbol{x}_0$ becomes a random variable in its own right instead of the limit of a noising process, and the *coupling* is free. Both are components of the library in Section 3.

Time sampling and conditioning. The derivation gives $t \sim \mathcal{U}[0, 1]$. However, the regression is not equally hard at every t, so reweighting puts capacity where the residual is large. The logit-normal schedule of Esser et al. [7] is the common choice. Conditioning again changes nothing structurally: we append $\boldsymbol{c}$ and hold it fixed while $\boldsymbol{x}$ evolves, since (1) moves the state and not the condition. Sampling is one draw and one solve of (1), which is where the step count becomes an accuracy/cost dial.

2.1.3 Rectified flows

Sampling cost is the number of function evaluations, and that number depends on how curved the trajectories are. A straight path travelled at constant speed is integrated exactly by a single Euler step. So it is worth asking how straight our trajectories actually are, and to ask it we first need a definition.

Definition 1 (Straightness; 23)**.** *For a flow with trajectories* $\boldsymbol{z}_t$*,*

$$S = \int_0^1 \mathbb{E}\left[\left\| (\boldsymbol{z}_1 - \boldsymbol{z}_0) - \boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{z}_t, t) \right\|^2 \right] \mathrm{d}t, \tag{14}$$

so $S = 0$ *exactly when every trajectory is a straight line at constant speed.*

Note what this measures: the deviation from the *chord*, not the spread of speeds. A field that turns at constant speed is curved, and it should not score as straight.

This raises a question. The conditional paths of (12) are already straight, so is the flow they induce straight as well? It is not, and the reason is that they *cross*. Two segments can pass through the same $(\boldsymbol{x}_t, t)$ carrying different velocities. The marginal field (9) takes one value there, namely the posterior average, and ODE trajectories cannot intersect at all. Hence the flow realises a different, non-crossing pairing along curved paths, and $S > 0$. Straight conditional paths do not give a straight marginal flow.

The reflow procedure. Liu et al. [23] replace the independent coupling with one that the model itself produces. Given a trained $\boldsymbol{v}_{\boldsymbol{\theta}}$, we draw $\boldsymbol{z} \sim \rho_0$, integrate (1) accurately to get $\boldsymbol{x}_1' = \mathrm{ODE}(\boldsymbol{z})$, and retrain on the pairs $(\boldsymbol{z}, \boldsymbol{x}_1')$. That coupling is deterministic and, because an ODE flow induces it, non-crossing. So the same objective can now be fitted by straight lines. Repeating gives the k-rectified flow. Marginals are preserved at every iteration, and the transport cost does not increase.

Remark 3 (Rectified flow is not the same thing as reflow)**.** *The two are routinely confused. Training with a linear path and an independent coupling, which is what people call "1-rectified flow", is mathematically identical to standard flow matching and straightens nothing. Reflow is the retraining procedure, and it is the part that produces the straightening. FlowPDE keeps the two separate (Section 3.2.3).*

Remark 4 (Reflow needs the original pairing)**.** *Reflow works only if training uses the* same $\boldsymbol{z}$ *that generated each target. Everything the procedure does depends on that pairing. If we draw fresh noise instead, we decouple the pairs and reduce reflow to training on the model's own samples, which straightens nothing. And the loss curve looks identical either way, so the bug is invisible in the training diagnostics. This is exactly why the source of* $\boldsymbol{x}_0$ *is an explicit component in Section 3, rather than an implicit* $\mathcal{N}(\mathbf{0}, I)$ *buried inside the loss.*

One last distinction. Reflow is not distillation. It retrains the same objective on a new coupling and keeps a valid ODE at every stage, whereas distillation fits a student to imitate a teacher's endpoints and gives up the ODE structure.

2.2 Application to parametrized PDEs

Nothing above mentions a differential equation. This subsection supplies the missing half. We answer what the distribution is over, and why one would want a distribution at all.

The parametrized problem. Let $\boldsymbol{\mu} \in \mathscr{P}$ be a parameter, either physical coefficients or a field such as a diffusion coefficient, a forcing term or an initial condition, and consider

$$\mathscr{A}\big(u; \boldsymbol{\mu}\big) = 0 \ \text{ in } \Omega, \qquad \mathscr{B}(u) = 0 \ \text{ on } \partial\Omega, \tag{15}$$

assumed well posed, so that the solution map $\mathscr{S} : \boldsymbol{\mu} \mapsto u(\boldsymbol{\mu})$ is single-valued. After discretization its image is the *solution manifold*: a set parametrized by $\mathscr{P}$, of intrinsic dimension at most $\dim \mathscr{P}$, sitting inside a space of dimension N_h that can run to millions.

That gap is the whole reason reduced order modelling exists, together with the fact that the applications needing it are *many-query* ones, such as optimization, control, inverse problems and uncertainty propagation [26].

The classical construction approximates the manifold by a linear subspace and closes the reduced problem by Galerkin projection. Its limitation is structural. Linear approximation succeeds only if the Kolmogorov n-width decays quickly, and for transport-dominated problems it does not, so n grows until the reduced model is no longer reduced. Deep-learning reduced order models replace the subspace with a learned nonlinear decoder [9], and neural operators learn $\mathscr{S}$ as a map between function spaces [18, 19].

Why a distribution. As Section 1 argued, these methods are deterministic, and under a mean squared error they return the conditional mean $\mathbb{E}[\boldsymbol{u} \mid \boldsymbol{\mu}]$. That is exactly right when $\mathscr{S}$ is a function, and it is an average over incompatible solutions when it is not. Our response is to model $p(\boldsymbol{u} \mid \boldsymbol{\mu})$ and sample from it.

Mapping this onto Section 2.1 needs no new machinery at all. The conditioning variable is $\boldsymbol{c} = \boldsymbol{\mu}$, the target is $\boldsymbol{x}_1 = \boldsymbol{u}_h$, and the trained field $\boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{x}, t; \boldsymbol{\mu})$ is an amortized sampler for the whole family $\{p(\cdot \mid \boldsymbol{\mu})\}$. That amortization is the offline/online split reached from the machine-learning side: one training run buys posterior samples for any new $\boldsymbol{\mu}$, at the price of a single ODE solve.

2.2.1 Forward problems

The forward problem is (15) read from left to right. Three of the benchmarks used later are of this type: the source-to-solution map $f \mapsto u$ for Poisson, the initial-to-final map for Burgers, and the coefficient-and-source map $(\kappa, f) \mapsto u$ for Darcy. They are drawn in the forward rows of Figure 2.

We should be clear about what this setting is. It is exactly the operator-learning task on which deterministic methods are strongest. The Fourier neural operator reaches around one per cent relative L^2 on the standard Darcy and Burgers benchmarks [19], so a generative surrogate that only matches it has shown nothing new.

Generative models are still applied to forward problems, but for reasons that are not about uncertainty. PDE-Refiner [22] observes that a mean squared error is dominated by the high-amplitude low-frequency part of the solution and effectively ignores the low-amplitude high-frequency part, which is what governs long rollouts, and it rewrites the update as a few denoising steps to force attention across the whole spectrum. Kohl et al. [17] find that autoregressive conditional diffusion keeps turbulent statistics faithful long after deterministic predictors have collapsed to over-smoothed fields. GenCast [25] does this at operational scale for weather. The common point is that where the pointwise map is unstable but its statistics are not, a distributional target is the better one.

Remark 5 (The forward posterior is a Dirac)**.** *For a well-posed forward problem $p(\boldsymbol{u} \mid \boldsymbol{\mu})$ is a point mass at $\mathscr{S}(\boldsymbol{\mu})$. Any spread that a conditional sampler shows is therefore* model error *and not physical uncertainty. Reporting it as uncertainty quantification would therefore be wrong, not just imprecise. All uncertainty claims of this report rest on Section 2.2.2, and the forward experiments are reported only as verification that the operator is learned at all.*

Given Remark 5, what is the forward setting still good for? Measurement. It has ground truth for every input, so the accuracy of the learned transport can be checked directly. And since sampling cost is the number of function evaluations, this is where the accuracy-against-cost curve can be traced on fixed weights. Straightness, reflow and solver choice are properly evaluated here. The probabilistic claims are not.

2.2.2 Inverse problems

The inverse problem reverses (15): we recover the data from measurements of the solution. Write the observation as

$$\boldsymbol{y} = \mathscr{O}\big(\mathscr{S}(\boldsymbol{\mu})\big) + \boldsymbol{\eta}, \qquad \boldsymbol{\eta} \sim \mathscr{N}(\mathbf{0}, \sigma^2 I), \tag{16}$$

with $\mathscr{O}$ typically a restriction to part of the domain. The Bayesian formulation [30] then looks for

$$p(\boldsymbol{\mu} \mid \boldsymbol{y}) \;\propto\; p(\boldsymbol{y} \mid \boldsymbol{\mu})\, p(\boldsymbol{\mu}), \tag{17}$$

and this is the right object precisely because the problem is ill-posed. Three mechanisms make it so: $\mathscr{O}$ discards information by construction, the noise destroys more of it, and $\mathscr{S}$ need not be injective.

The result is a posterior with real width, not a nearly degenerate one that a point estimate would approximate well enough. So reporting only its mean throws the answer away.

Darcy flow makes this concrete. The coefficient κ enters $-\nabla\cdot(\kappa\nabla u) = f$ only through the flux, so wherever ∇u is small the data barely constrain it, and many coefficient fields explain the same observation. Meaning that identifiability *varies over the domain*. A set of posterior samples represents that structure. A conditional mean flattens it into one smooth field, whose smoothness is an artefact of the averaging.

Now, how do we avoid the cost of the classical route? Markov chain Monte Carlo needs a forward solve for every likelihood evaluation, and the whole chain must be rerun for each new observation. There are two ways around this.

Route (a). Train an unconditional model of $p(\boldsymbol{u})$ and steer its sampler towards agreement with $\boldsymbol{y}$ at inference time [4]. One prior then serves any observation operator. But each new observation costs a full guided sampling run with likelihood gradients, and the guidance term is an approximation whose error is hard to control.

Route (b). Train a conditional model directly on pairs $(\boldsymbol{y}, \boldsymbol{\mu})$ from the simulator, learning $p(\boldsymbol{\mu} \mid \boldsymbol{y})$ outright. This is simulation-based inference: it needs samples from the forward model, but never its likelihood. Wildberger et al. [33] show that the objective of Theorem 1 gives posterior estimators scaling better than the normalizing-flow architectures used before. Its cost is that the model is tied to the observation operator and the noise level it was trained on.

FlowPDE takes route (b), and the reason is that amortized inference *is* the offline/online split of reduced order modelling. One expensive offline stage buys an online stage in which a new observation yields posterior samples through a single ODE solve, with no likelihood evaluation and no gradient through the physics. Route (a) has no online stage in that sense. This choice has a cost, and it is not an obvious default.

Checking that the posterior is right. A model that returns a distribution has to be scored as one. This point is often skipped in the literature, where relative L^2 of the ensemble mean is frequently the only number reported. That quantity measures accuracy and says nothing about calibration. A model can produce diverse, individually plausible samples whose spread has nothing to do with its actual error, and still score well. Three families of diagnostics close the gap, and Section 3 implements all of them.

Proper scoring rules. These reward accuracy and calibration together, and cannot be improved by misreporting uncertainty [10]. The continuous ranked probability score (CRPS) is the pointwise standard, and it reduces to mean absolute error for a deterministic forecast. Its multivariate version, the energy score [31], is the one that matters here. Pointwise scores are blind to spatial correlation, so a model that reproduces every marginal correctly while generating spatially incoherent fields scores well pointwise and badly under the energy score. Since PDE solutions are smooth and strongly correlated, that is exactly the failure we want to catch.

Calibration diagnostics. These ask whether the stated uncertainty matches the observed error. Coverage and the reliability curve compare nominal against empirical coverage. The rank histogram [12] plots where the truth falls among the members, where flat means calibrated, U-shaped means under-dispersed and dome-shaped means over-dispersed. The spread–skill ratio [8] compares the spread against the ensemble mean's error, with a finite-ensemble correction $\sqrt{(K+1)/K}$ applied before it is read.

Decomposition. This splits the predictive variance into an aleatoric part, which is one model's spread and the posterior width on a genuine inverse problem, and an epistemic part, which is the disagreement between independently trained models. It is what makes Remark 5 usable in practice: on a forward problem the aleatoric component should be zero, so any spread we do observe is really epistemic error.

3 FlowPDE

Section 2 described a design space. This section describes the software we wrote to explore it.

`flowpde` is a PyTorch library for training and sampling conditional continuous-time flows on PDE data. Its pieces are *separate*. The dynamics, the objective that fits them, the network that parametrizes them, the data, and the integrator that samples them are five different concerns, and each one can be replaced without touching the others. Listings below are quoted from the source, shortened, with the cuts marked.

3.1 Library overview

The design goal is best stated by example. Listing 1 is a complete training and sampling run on the 2D Darcy forward problem. There is no configuration file, no framework, and no per-problem training script.

```
from flowpde import NeuralODEFlow, FlowMatchingObjective, UNet, Trainer
from flowpde.datasets.exponax import DarcyGenerator
from flowpde.datasets import FieldNormalizer
from flowpde.trainers import FlowEvaluator

# 1. Data: (kappa, f) -> u on a 64x64 grid.
gen = DarcyGenerator(num_points=64, kappa_alpha=2.0, kappa_tau=3.0)
train_ds = gen.generate(num_samples=1000, seed=0, problem='forward')
val_ds = gen.generate(num_samples=200, seed=1, problem='forward')

# 2. Normalization: fit on train, share the SAME instance with val.
normalizer = FieldNormalizer.from_dataset(train_ds)
train_ds.set_normalizer(normalizer); val_ds.set_normalizer(normalizer)

# 3. Velocity field, dynamics, objective.
model = UNet(spatial_dim=2, spatial_size=64,
             solution_channels=1, condition_channels=2, base_channels=64)
flow = NeuralODEFlow(model, target_key='target', condition_key='input')
objective = FlowMatchingObjective(flow, path='linear', time_sampler='uniform')

# 4. Train, selecting on sampled error rather than on the loss.
evaluator = FlowEvaluator(objective, val_loader, normalizer=normalizer,
                          target_fields=val_ds.target_fields)
trainer = Trainer(objective, torch.optim.Adam(model.parameters(), lr=1e-3),
                  ema_decay=0.999, validator=evaluator, monitor='rel_l2',
                  checkpoint_extra={'normalizer_state': normalizer.state_dict()})
trainer.train(train_loader, epochs=200, save_dir='runs/darcy')

# 5. Sample: n_steps is the accuracy/cost dial of Section 2.1.3.
u = objective.sample(condition=batch['input'], n_steps=50, solver='euler')
```

Listing 1: A complete run. Every line is a modelling choice; none is boilerplate.

The central separation. The important structural point is in step 3. `NeuralODEFlow` and `FlowMatchingObjective` are two objects and not one. This is Remark 1 made concrete.

The flow owns the *dynamics*, namely the velocity field, the integration, and the exact log-likelihood by (4). The objective owns the *training procedure*. So when we swap `FlowMatchingObjective(flow)` for `MaximumLikelihoodObjective(flow)`, we change how the same field is fitted and nothing else: same model, same data, same normalization, same sampler.

This means the comparison between simulation-free and simulation-based training is a controlled experiment rather than a comparison between two codebases, and that is what Section 4 uses. The reverse rule is enforced too: no training logic in the flow, and no dynamics in an objective.

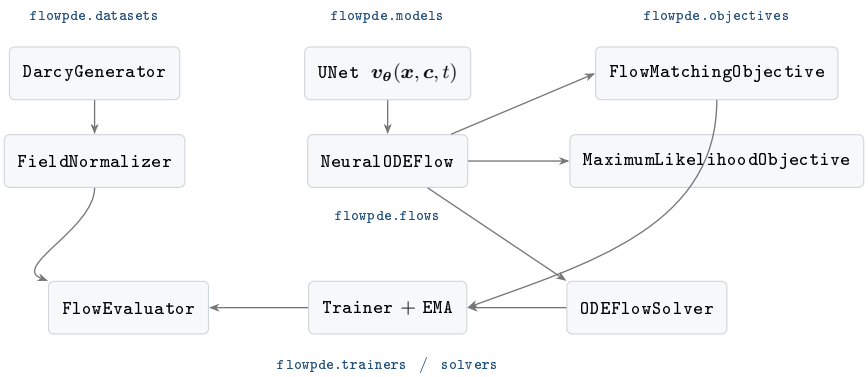

Fig. 1. Module dependencies. Arrows point from a component to the one that uses it. The flow depends on a network and nothing else, the objectives depend on the flow, and the trainer depends only on an object exposing `compute_loss` and `model`, so it does not care which objective it is given.

3.2 Modules and implementation details

We now go through the modules one by one, in the order in which they depend on each other.

3.2.1 `flowpde.core`

This module holds three abstract bases, `BaseFlow`, `BaseSolver` and `BaseConditioner`, plus an optional configuration layer that the core itself never imports.

The most important thing `BaseFlow` provides is a tensor contract rather than an abstract method. `_extract_target_condition` flattens target and condition to (B, d) before anything else sees them, so a field on a 64×64 grid reaches the objective as a vector of length 4096, and the convolutional backbones reshape it back internally.

The benefit is that flow, objective, path, coupling and solver are all written once in $\mathbb{R}^d$, with no special handling of spatial structure. The cost is that shapes must be declared to the backbone at construction, and Remark 6 returns to that.

`BaseConditioner` abstracts *how* the condition enters the network. `ConcatConditioner` concatenates $\boldsymbol{c}$ to $\boldsymbol{x}$ along channels, `FiLMConditioner` modulates features by $\gamma(\boldsymbol{c}) \odot h + \beta(\boldsymbol{c})$, and `NullConditioner` discards the condition entirely. That last one is not a placeholder. It is the control variant: a model that cannot see the PDE parameters sets the floor that any conditional model has to clear.

3.2.2 `flowpde.models`

Four backbones share one signature, `forward(x, f, t)`, and reshape internally because the flow flattens. `MLP` is fully connected, for problems where spatial structure carries no information. `ConvNet` is a stack of time-conditioned residual convolutions at fixed resolution. `ResNet` [13] deepens that stack. `UNet` [28] adds encoder–decoder skips with optional bottleneck attention and a depth derived from the resolution,

$$\min\bigl(5, \lfloor \log_2 S \rfloor - 2\bigr),$$

so downsampling stops at four to eight pixels rather than at one.

Two details recur across all four. Time enters through a sinusoidal embedding added residually *inside each block*, so that every level is time-aware. And the output projection is *zero-initialized*, so a freshly constructed backbone predicts exactly $\boldsymbol{v}_{\boldsymbol{\theta}} \equiv \mathbf{0}$. The second one is standard for flow models, since the flow starts as the identity, but it has a practical consequence: a test that perturbs the input of a fresh model and expects the output to change will fail until `output_proj.weight` is perturbed first.

3.2.3 `flowpde.flows`

`NeuralODEFlow` implements Section 2.1.1. It integrates (1) through `torchdiffeq`, and computes the exact conditional log-likelihood by augmenting the state with $\log p$, using either an exact trace at d Jacobian-vector products or the Hutchinson estimator (6) at one.

One interaction is worth noting. The likelihood needs higher-order derivatives through the trace estimator, and the adjoint method's custom backward solve does not suit that graph, so the implementation disables the adjoint whenever `compute_logp` is set.

Remark 6 (A known rough edge)**.** *The flattening contract means the flow has to be told the target dimension when it differs from the condition dimension, as it does for Darcy, where the condition is $[\kappa, f]$ on two channels and the target is u on one. At present `flow._target_dim` is set as a side effect of `compute_loss`, so a checkpoint loaded fresh for inference has never had it set, and `sample()` quietly falls back to the condition dimension. The workaround is to pass `target_shape=` explicitly, and the fix is to accept the shape at construction. We record it here rather than leave it out, since a report describing its own software should say where the software is wrong.*

3.2.4 `flowpde.objectives`

`FlowMatchingObjective` composes the four components of Section 2.1.2. Each is accepted either as a registry name or as a constructed instance, which makes the API config-friendly without making configuration a dependency. Its entire training step is Listing 2, and it corresponds to (10) line by line.

```
def compute_loss(self, batch, target_key=None, condition_key=None, **kwargs):
    x_1, condition = self.flow._extract_target_condition(batch, ...)

    # Passing the batch lets BatchSource return the precomputed x_0
    # that reflow depends on, instead of drawing fresh noise.
    x_0 = self.sample_base_distribution(x_1.shape, self.model_device, batch)
```

```
# A source carrying its own pairing already fixes which x_0 goes with
# which x_1, so re-coupling here would destroy it.
if not self._source_defines_pairing(batch):
    x_0, x_1 = self.coupling(x_0, x_1)

t = self.time_sampler(batch_size, self.model_device)
x_t, v_target = self.path(x_0, x_1, t) # interpolate + velocity
v_pred = self.model(x_t, condition, t)
return F.mse_loss(v_pred, v_target)
```

Listing 2: `FlowMatchingObjective.compute_loss`: (10) in eight lines.

Table 1 lists what may be substituted at each slot. The two guarded branches in the listing encode the two conditions of Remarks 2 and 4. The path returns interpolant and velocity *together*, so the two cannot disagree, and the coupling is skipped when the source already defines the pairing, so minibatch-OT cannot quietly reorder reflow's pairs. `BatchSource` is `strict=True` by default, meaning a missing key raises an error rather than resampling.

Slot	Options	What it controls
Path	`linear`, `ot_conditional`	the interpolant and its velocity
Time	`uniform`, `logit_normal`, `beta`, `truncated`	where capacity is spent in t
Coupling	`independent`, `minibatch_ot`	which $\boldsymbol{x}_0$ meets which $\boldsymbol{x}_1$
Source	`gaussian`, `batch`	where trajectories start

Table 1. The four flow-matching slots. Each accepts a registry name or an instance, so the variants of Section 2.1.2, namely standard flow matching, rectified flow, OT-CFM and minibatch-OT coupling, are configurations rather than subclasses.

The second objective, `MaximumLikelihoodObjective`, fits the same flow by maximizing (5) instead. It shares the model, the data and the sampler, which is what makes the comparison in Section 4.2 a controlled one.

3.2.5 `flowpde.trainers`

`Trainer` accepts any object exposing `compute_loss(batch)` and a `.model`, which is why it depends on neither objective specifically. Two of its features are less standard and more important.

Exponential moving averaging. The flow-matching gradient is noisy for a structural reason. By Theorem 1 the regression target is only conditionally determined by the network's input, so the iterates bounce around the optimum instead of settling into it. Averaging suppresses that, and the averaged weights are what we evaluate and ship. A warmup ramp, $\min(d, (1+n)/(10+n))$, removes the need for a "start averaging at step N" threshold.

Selecting on sampled error, not on the loss. This is the most important choice in the whole module. As Section 2.1.2 explained, the flow-matching loss has a large irreducible floor, so two checkpoints can differ a lot in sample quality while their losses differ only in the fourth decimal. `FlowEvaluator` therefore integrates the ODE, denormalizes, and scores against ground truth, and `Trainer` selects on that. Evaluation noise uses a *fixed seed*, so differences from one epoch to the next reflect the model and not the draw. With `ensemble_size > 1` it scores the ensemble mean and reports the spread, and this is where the diagnostics of Section 2.2.2 attach: `flowpde.utils.uq_metrics` implements coverage, reliability curves, rank histograms, spread–skill with the $\sqrt{(K+1)/K}$ correction, CRPS, the energy score, and the aleatoric/epistemic decomposition.

Reflow, and what enforcing its condition bought us. Remark 4 states the correctness condition. Training must use the same $\boldsymbol{x}_0$ that generated each target, and `BatchSource` is `strict=True` so that a missing key raises rather than quietly resampling. We tested that machinery on the Burgers ConvNet at three seeds and two reflow iterations, against a control that deliberately resamples $\boldsymbol{x}_0$ independently, which is the bug the rule exists to prevent.

Two results came out, and both are negative. The correct procedure does straighten, from 8.23 to 5.18. But *so does the broken one*, from 8.23 to 5.81. Retraining on the model's own samples apparently straightens on its own, which tells us that straightening is not by itself evidence that the pairing was honoured. And neither arm improved few-step accuracy: one-step relative L^2 moved from 0.0996 to 0.1048 (paired) and 0.0999 (decoupled), and the error rose at every larger budget. On this problem the baseline was already accurate at eight evaluations, so there was little for reflow to recover and two iterations of retraining to lose.

The condition is still a correctness condition and is still enforced by the structure. What the experiment shows is that its *observable* effect is smaller and less useful as a diagnostic than the literature suggests.

3.2.6 `flowpde.datasets`

Data generation runs through Exponax [16] on JAX [2]. `PoissonGenerator` solves $\nabla^2 u = f$ spectrally. `BurgersGenerator` advances an initial condition with an exponential time-differencing stepper, and can draw a per-sample viscosity $\nu \sim \text{LogUniform}$, though the experiments of Section 4 fix $\nu = 10^{-2}$. `DarcyGenerator` solves $-\nabla\cdot(\kappa\nabla u) = f$ with second-order finite differences and a fixed-step conjugate gradient written through `jax.lax.scan`, so that the pipeline is `jax.vmap`-able with no Python loop over samples. Here κ is a log-normal Gaussian random field,

$$\kappa = \exp(c\,g), \qquad g \sim \mathcal{N}\big(0, (\tau^2 I - \Delta)^{-\alpha}\big), \tag{18}$$

which guarantees $\kappa > 0$ everywhere and reproduces the FNO Darcy benchmark at $\alpha = 2$, $\tau = 3$ [19].

Every generator takes `problem='forward'` or `'inverse'`, which swaps input and target, and the inverse direction turns on additive observation noise (`obs_noise_std`) and random spatial masking (`obs_mask_fraction`). Together these implement (16) and make posterior degeneracy a controlled variable.

One detail here is worth explaining. We *keep* the mask rather than throwing it away, because a masked observation is otherwise ambiguous: a zero may mean "the field is zero here" or it may mean "nothing was measured here". So the mask is appended as an extra conditioning channel, and the model must be built with `condition_channels` matching `sample['input'].shape[0]`.

Normalization. Flow matching transports $\mathcal{N}(\mathbf{0}, I)$ to the data, so a target with non-unit scale makes the velocity regression (10) badly conditioned. `FieldNormalizer` standardizes each field, and two of its decisions matter.

First, statistics are keyed by *raw field name*, such as `'source'`, `'solution'` or `'kappa'`, and not by the role the field plays. This is why one normalizer stays valid across `'forward'` and `'inverse'`, where the roles swap but the fields themselves do not.

Second, a field with no registered statistics passes through untouched, which keeps `obs_mask` binary instead of standardizing a 0/1 indicator into something meaningless.

Metrics are therefore reported in physical units. Predictions are mapped back with `denormalize_channels`, and `FlowEvaluator` refuses to run with a normalizer but no `target_fields`, rather than quietly scoring in standardized units.

Remark 7 (Never refit on validation)**.** *`FieldNormalizer.from_dataset(train_ds)` is called once, and the resulting* instance *is shared with validation and test. Fitting a second normalizer on held-out data leaks its statistics into the evaluation. The API makes the correct usage the short one, but it cannot make the incorrect one impossible.*

3.2.7 `flowpde.solvers`

`ODEFlowSolver` wraps `torchdiffeq` and exposes fixed-step methods (`euler`, `midpoint`, `rk4`) together with adaptive ones (`dopri5` [6], `tsit5`, and others). The condition is held fixed while $\boldsymbol{x}$ evolves, which implements the rule that (1) moves the state and not the conditioning variable.

Solver and step count are chosen at call time on trained weights. That is what makes the accuracy-against-cost curve of Section 4.3 measurable without retraining anything, and it is the reason the solver is a module of its own rather than an argument buried inside the flow.

3.3 Installation and usage

`flowpde` targets Python 3.11 and depends on `torch` [24], `torchdiffeq`, and `jax`/`exponax` [2, 16]. The environment is managed with `uv`: `uv venv && uv sync`, then `uv run -m pytest`. Listing 1 is a complete working example.

The test suite covers components, models, solver, normalization, trainer, EMA, evaluation, straightness, reflow and the UQ metrics. Wherever an exact answer exists, we test against that instead of against a stored baseline. Path velocities are checked against finite differences of the interpolant, the solver against $\mathrm{d}\boldsymbol{x}/\mathrm{d}t = -\boldsymbol{x}$, the straightness of $\boldsymbol{v} = 2t\boldsymbol{c}$ against its exact value of $1/3$, and the EMA recursion against hand-computed arithmetic. Tests that invoke the Exponax solvers are marked `slow`.

4 Numerical experiments

This section tests the claim of Section 2.2: that a conditional flow works as a reduced model of a parametrized solution manifold. We first check that the map is learned at all, then treat the sampling cost

as a knob, then look at what the inverse direction returns, then control how ill-posed the problem is, and finally compare against MCMC.

One rule holds throughout. Every comparison varies *one* axis and holds everything else fixed. Data, splits, normalization, optimizer, schedule, averaging, sampler and evaluation seed are shared, so a difference between two numbers can be attributed to the thing named in the column heading. Where we failed to keep that rule we say so, and Section 4.7 collects the cases.

Two earlier points also limit what we may claim. Remark 5 says we may not read sampler spread on a forward problem as uncertainty, and Section 2.2.2 says we must score a distribution as a distribution.

4.1 Setup

Benchmarks. We use three PDE families. *Poisson*, $\nabla^2 u = f$, is learned forward at 64×64 from a random sine source, at two spectral widths that we discuss below. *Burgers* in 1D maps an initial condition to the state at $T = 0.4$ at fixed viscosity $\nu = 10^{-2}$. Both are also run in the inverse direction under observation noise in Section 4.5, with Poisson at 32×32.

Darcy in 2D, $-\nabla \cdot (\kappa \nabla u) = f$ on $[0,1]^2$ at 64×64, carries most of the experiments. There κ is the log-normal field (18) at $\alpha = 2$, $\tau = 3$, which are the FNO benchmark parameters [19], the source is a random Fourier field with cutoff 8, and the linear system is solved by 500 conjugate-gradient iterations.

Each PDE is available in both directions as an argument on the same generator, which gives the six settings of Figure 2.

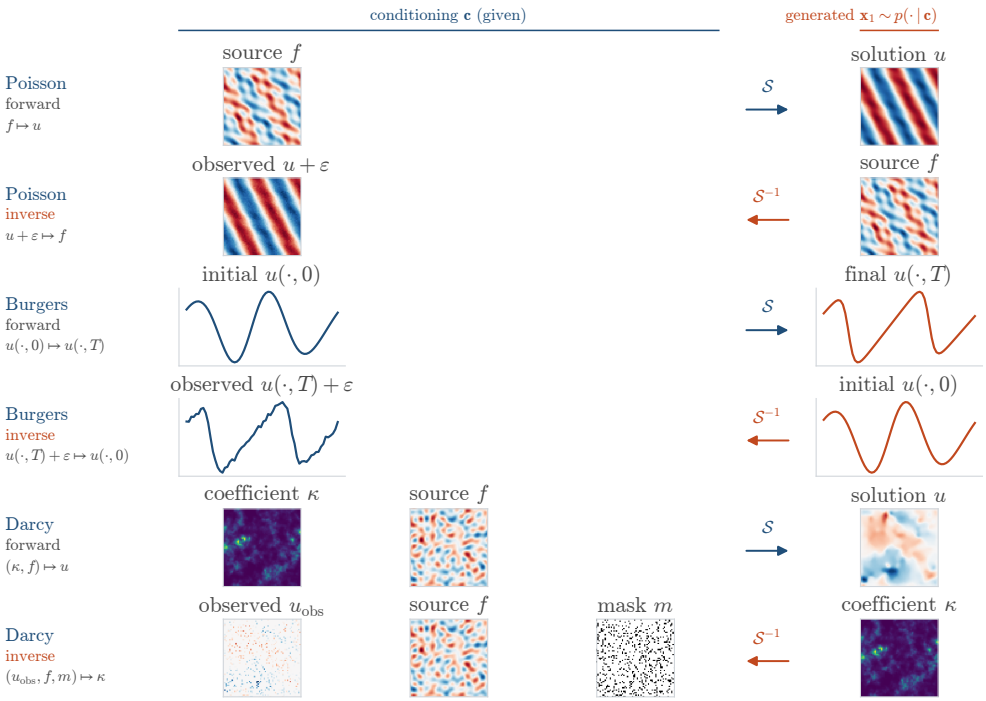

Fig. 2. The six problem settings. Each row is one conditional distribution the library trains: everything left of the arrow is conditioning, stacked as channels of a single input tensor, and the field on the right is what the flow generates. Forward rows apply the solution operator $\mathscr{S}$, inverse rows invert it. The inverse rows show the observation the model is actually given: Poisson and Burgers under additive noise, Darcy at 15% observed with its mask carried as a third conditioning channel. Poisson is drawn in the widened source regime. Fields are from the seeded test generators, so they are the distributions the models below were scored on.

How much of an error belongs to the benchmark. Before comparing anything, we should ask how much of a reported number comes from the method and how much comes from the data it was measured on. E8 tests this.

The Poisson source is a random sine series with n terms at wavenumbers up to $k_{\max}$, and the generator's default, $n = k_{\max} = 3$, spans a nine-dimensional subspace. Widening it to $n = k_{\max} = 8$ moves relative L^2 from 0.054 to 0.078 for the ConvNet, and from 0.061 to 0.066 for the UNet.

So a 43% increase for the better model came from changing the distribution alone. An operator-learning number therefore says something about the method and about the data together, and for that reason we

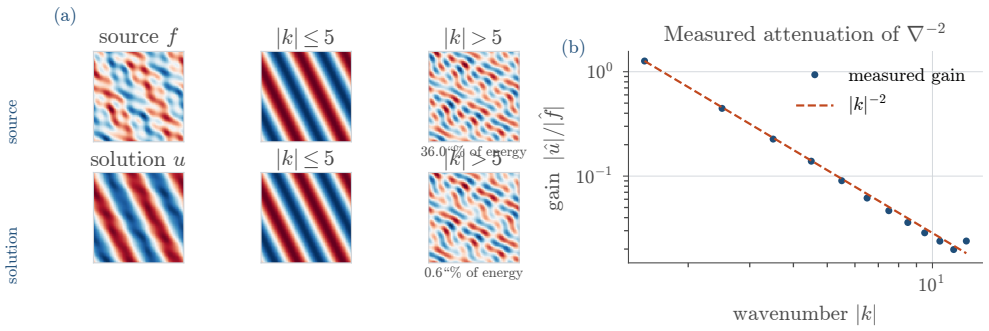

Fig. 3. Why the inverse direction is the harder one. The forward Poisson operator has spectral gain $|k|^{-2}$, so inverting it multiplies the high-wavenumber band by $|k|^2$, and that band is where observation noise lives. The parametrization is recoverable only to the extent that the observation constrains it, which is what Section 4.5 turns into a controlled variable.

quote the harder regime whenever there is a choice.

What is held fixed. Every run trains a conditional `NeuralODEFlow` on the standard flow-matching configuration, meaning a linear path, a uniform time sampler, an independent coupling and a Gaussian source, with AdamW at 2×10^{-4}, cosine decay, gradient clipping at 1.0, and EMA at $\rho = 0.999$. Splits are 3000/200/200 at fixed generator seeds, so every variant sees identical data. One `FieldNormalizer` is fitted on train and shared (Remark 7). Selection is on sampled validation error at a fixed seed and never on the training loss. Table 2 records what each experiment varies.

	Problem	Varied	Levels	Epochs
Section 4.2: conditioning, backbone, objective				
E1	Darcy 2D, fwd.	conditioning	null, concat	500
E2	Burgers 1D, fwd.†	backbone	ConvNet, ResNet, UNet, UNet$^-$	500
E3	Darcy 2D, fwd.	backbone	ConvNet, ResNet, UNet, UNet$^-$	500
E8	Poisson 2D, fwd.	backbone, source spectrum	2×2	150
E4	Darcy 2D, fwd. 16^2†	objective	flow matching, max. likelihood	50
Section 4.3: sampling cost				
E7	Burgers 1D, fwd.†	path, schedule, coupling	6 configurations	500
Sections 4.4 and 4.5: posteriors				
E5	Darcy 2D, inverse	conditioning, target	null/concat $\times$ $\kappa/(\kappa, f)$	100, 500
E6	Darcy 2D, inverse	backbone	ConvNet, ResNet, UNet, UNet$^-$	500
E9	Darcy 2D, inverse	observed fraction, noise	$4 + 4$ levels	100
E11	Poisson 2D, inv. 32^2	observation noise	4 levels	500
E12	Burgers 1D, inverse	observation noise	4 levels	500
Section 4.6: comparison against MCMC				
E10	Darcy 2D, inverse	sampler: pCN reference	4 obs. $\times$ 2 chains	—
E13	Burgers 1D, inv. 256	sampler, identifiability	2 regimes $\times$ 4 obs.	150, 500
Re-evaluations on finished weights, no retraining				
R1	Burgers, Darcy, Poisson	solver, step count	4 solvers, 8 step counts	—
R2	Burgers, Darcy	straightness mode	trajectory, interpolant	—
R3	Darcy, both directions	ensemble size	$K = 32$	—

Table 2. The experiment matrix. UNet$^-$ is the UNet without attention. † marks the experiments repeated at three matched seeds (42, 1042, 2042) per cell, and the rest are single runs. R1–R3 are measurements on finished weights, which need no retraining because sampling cost, straightness and ensemble size are all chosen at inference time.

Metrics. Errors are relative L^2 in physical units on the held-out split, with relative H^1, MAE and a normalized maximum error recorded alongside. Distributions are scored with CRPS, the energy score, coverage and the bias-corrected spread–skill ratio over $K = 32$ draws at a fixed ensemble seed.

One point about relative L^2 has to be settled before any inverse number is read, so we state it as a remark.

Remark 8 (Relative L^2 near one is not a small number)**.** *On the inverse problems below, several models score relative $L^2 \approx 1$. That is not near-perfect agreement with a differently scaled truth. It is roughly what using none of the conditioning information achieves, and there are two such levels rather than one.*

A deterministic *model that ignores its input and returns the mean field scores exactly* 1. *A* generative *model that ignores its input and draws from the marginal returns an independent sample, so for a zero-mean field*

$$\mathbb{E}\,\|\hat{\boldsymbol{u}} - \boldsymbol{u}\|^2 = 2\,\mathbb{E}\,\|\boldsymbol{u}\|^2$$

and the score is about $\sqrt{2}$.

The gap between the two is not a defect of the generative model. It is the price of returning a sharp admissible field instead of a blurred average, and it is why a single draw must not be scored as if it were a point estimate. Section 4.4 therefore keeps an unconditional model in every comparison, so that the reference level is measured on the same benchmark and not assumed.

Hardware. Training on one CUDA device for E1–E10, and on Apple silicon for E11–E13. Re-evaluations (R1–R3), solver timings and the MCMC chains ran on CPU. Wall-clock numbers come from more than one machine, so they are comparable only within a table.

4.2 Conditioning, backbone and objective

Before anything else we check two things: that the conditioning tensor, which holds the PDE parameters, really reaches the velocity field, and that the resulting map is accurate. Both are verification rather than contribution, so we report them briefly.

The conditioning is actually used. A conditional flow can fail without any visible sign. If the condition never influences the velocity, the model still learns a good *unconditional* generator of Darcy solutions, its loss still decreases, and its samples still look like plausible fields. E1 replaces `ConcatConditioner` with `NullConditioner` and changes nothing else.

On the same 200 test samples the conditioned model is better on *every one*, with a median per-sample ratio of 17.2, and median relative L^2 falls from 1.30 to 0.092. Both loss curves converge and both look healthy.

Two things follow. First, this is the concrete reason we select on `FlowEvaluator` rather than on the loss. Second, it gives us a scale for the rest of the section, since the null model is an honest unconditional sampler, so 1.30 is what learning nothing looks like here.

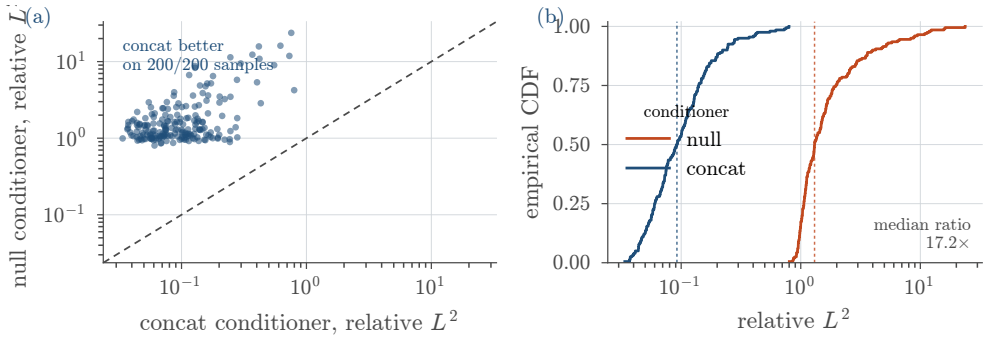

Fig. 4. Conditioning on Darcy 2D, 200 paired test samples. **(a)** Per-sample relative L^2 of the null conditioner against the concat conditioner; every point lies above the diagonal. **(b)** The same data as empirical CDFs, with dashed lines at the medians. Both models minimize the same objective and both loss curves converge, so the difference is invisible without sampling.

The smallest backbone is the most accurate. E2, E3 and E8 vary only the network parametrizing $\boldsymbol{v}_{\boldsymbol{\theta}}$. On Burgers the ConvNet at 0.31M parameters reaches relative L^2 0.021, against the UNet's 0.266 at 2.34M, so a factor of 12.4 in favour of the smaller network. On Darcy the ConvNet at 0.73M reaches 0.073 against 0.128 at 5.70M, and it is better on 176 of 200 individual test samples. Attention changes nothing: UNet and UNet$^-$ agree to within 1.5% on Darcy and to within seed noise on Burgers.

The obvious objection is that a single run per cell measures seeds and not architectures. So we reran E2 at three *matched* seeds, which gives ConvNet 0.0214±0.0003, ResNet 0.1420±0.0026, UNet 0.2657±0.0017 and UNet$^-$ 0.2660 ± 0.0013. A seed spread of 0.5–1.8% against a 12.4× gap settles the ordering, and it settles the attention question too, since the two UNets differ by five times *less* than the seed noise.

So at this problem size, searching over architectures is not a good use of effort.

There is one place where the ordering reverses. On E8's widened Poisson source the UNet wins by 15%, because its multi-scale structure helps once the source spans more modes. One other thing should

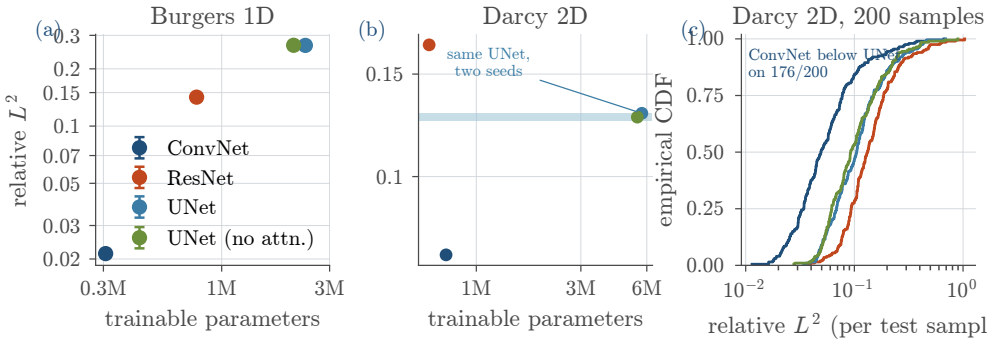

Fig. 5. (a, b) Accuracy against parameters; the ordering is inverted with respect to capacity. Error bars in (a) are the standard deviation over three matched seeds, and they are smaller than the markers. The band in (b) spans two runs of the *identical* UNet at different seeds, which is the only replicate available on Darcy. **(c)** Per-sample errors on Darcy.

be recorded. On Burgers, above $|k| = 13$, every model is wrong by a factor between 48 and 400, in a band that carries too little energy to show up in relative L^2. That is the observation behind PDE-Refiner [22], and here we measure it rather than assume it.

Flow matching against maximum likelihood. E4 fits one flow, one UNet of 40,777 parameters, one dataset and one sampler in two ways, changing only `FlowMatchingObjective` for `MaximumLikelihoodObjective`. The problem is deliberately small, Darcy at 16×16, because the simulation-based route is not affordable at 64×64.

The result is that each objective wins on its own metric. Maximum likelihood reaches -1.86 nats per dimension against -1.21. Flow matching reaches relative L^2 0.610 against 1.064, and it does so at 1.92 seconds per epoch against 135.2, a factor of 70, using 18.6 MB of peak memory against 166.3.

The replication adds one more thing. Across three matched seeds flow matching varies by 4.2% and maximum likelihood by 39.3%, its three runs scoring 0.745, 0.909 and 1.538. The worst of them is worse than the prior, from nothing but the draw, so a single-run comparison here would have reported whichever number it happened to get.

So the conclusion is narrower than "flow matching is better". At a fixed small budget the simulation-free objective gives better samples and more reproducible runs, the simulation-based one gives better density estimates, and the difference in compute between them is two orders of magnitude. Everything below uses flow matching for that reason.

4.3 Accuracy against sampling cost

Sampling is an ODE solve, so the evaluations spent per solution are chosen *after* training, on frozen weights. There is one offline cost and a family of online costs. R1 sweeps that family, R2 measures the path geometry behind it, and E7 shows that this geometry is itself a configuration string.

Throughout, we plot error against *function evaluations* and not against steps, because one RK4 step costs four evaluations and a per-step comparison would make RK4 look four times better than it is.

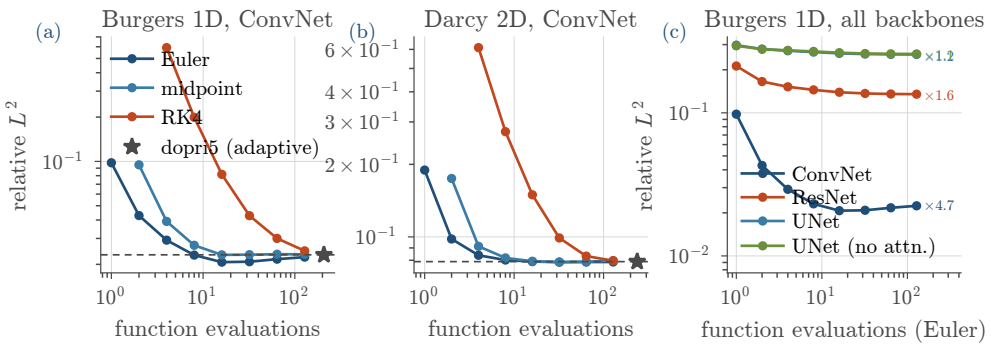

Fig. 6. Accuracy against cost on frozen weights. **(a, b)** The best backbone per problem; the star and dashed line mark the converged solution of the learned ODE. **(c)** Euler on every Burgers backbone. The *slope* is the diagnostic: steep means discretization error dominates, flat means the field itself is the limitation.

A few steps are already enough. The ConvNet's Burgers error falls by 4.7× from one evaluation to its best, and it reaches within a few per cent of converged accuracy by eight to sixteen evaluations. Figure 7 shows why the cheap end works at all. A *single* Euler step, which is a straight-line jump from noise along the learned velocity at $t = 0$, already recovers the structure of the solution.

This means the accuracy and the latency can be traded at call time instead of at training time, which is the offline/online split of Section 2.2 working as intended.

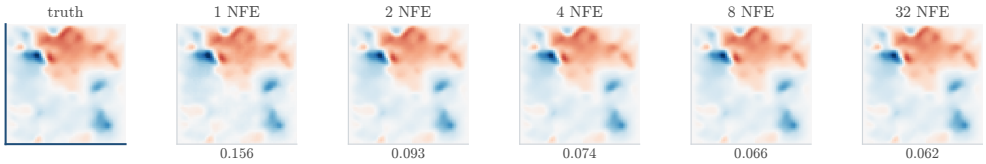

Fig. 7. One Darcy test condition, one fixed $\boldsymbol{x}_0$, integrated with Euler at increasing budgets; the number below each panel is that sample's relative L^2. A single evaluation already recovers the structure of the solution, which is what makes the left end of Figure 6 usable.

Euler beats the higher-order solvers at equal cost. On Burgers with the ConvNet, Euler and RK4 at the same budget differ by 8.6× at 8 evaluations (0.0231 against 0.1992), by 3.9× at 16 and by 2.1× at 32. On Darcy the ratios are 3.4, 1.9 and 1.3, and every backbone on both problems has RK4 at or behind Euler below 64 evaluations.

Why does the classical ordering fail here? Because a high-order method gets its accuracy by evaluating the field *off* the trajectory. A single RK4 step across $[0, 1]$ evaluates $\boldsymbol{v}_{\boldsymbol{\theta}}$ at $\boldsymbol{x} + h\boldsymbol{k}_3$, far from anything seen during training, and a learned field has no accuracy guarantee there. Classical order counts terms in a Taylor expansion of the true field, but here the field itself is only an approximation.

Converging the ODE is not the goal. The ConvNet on Burgers is best at 0.0207 with sixteen Euler evaluations. Integrating the same weights to convergence (`dopri5`, 208 evaluations) gives 0.0232, which is 11% *worse*. Coarse Euler lands closer to the data than the exact solution of the learned ODE does, because its truncation error partly cancels the model's own error.

The effect appears again elsewhere but not everywhere. On Darcy the ResNet gains 6% this way, while the two UNets match their converged error to four decimals. So we only state the negative version, which is enough for our purposes: the converged solution of the learned ODE is not special, and reporting accuracy "at 50 steps" gives one point on a curve that is neither monotone nor the same from model to model.

The time schedule is what straightens the trajectories. Section 3.1 claimed that the published flow-matching variants are *configurations* of one objective and not subclasses. E7 tests that claim with six configurations reached by changing registry strings, on one Burgers ConvNet, at three matched seeds each.

Logit-normal sampling [7] cuts trajectory straightness from 8.17 to 3.70, and the beta sampler cuts it to 4.95. The payoff appears where we would expect it. At two evaluations logit-normal reaches 0.0366 against the uniform baseline's 0.0467, and at four it reaches 0.0246 against 0.0313, so 22% better in both cases against a seed spread of 1.6%.

But it is also the *worst* non-OT configuration once the budget is adequate, at 0.0280 against 0.0214, a 31% loss, and the curves cross near twelve evaluations. So straightening is not a free improvement. It moves error from the few-step regime into the converged one, and which side of the crossing we sit on decides whether it is worth doing.

Two negative results belong here as well.

Optimal-transport coupling did not pay. Minibatch-OT coupling and OT-CFM are worse than the baseline at *every* budget, at 0.0375 and 0.0372 converged against 0.0214, which is the opposite of what we expected. Figure 8(b) shows what happened. The OT coupling produces by far the straightest *interpolant*, 1.28 against 1.66, while leaving the model's own trajectories as curved as the baseline's. It straightened the training target and not the learned dynamics. This is why Definition 1 is stated on trajectories, and why a report using interpolant straightness alone would have concluded that OT coupling worked.

Reflow straightened even when it was broken. For the same reason, Section 3.2's reflow experiment straightened under a deliberately broken pairing almost as much as under the correct one, 8.23 → 5.81 against 8.23 → 5.18, and improved few-step accuracy in neither arm. Straightening is not by itself evidence that a method worked.

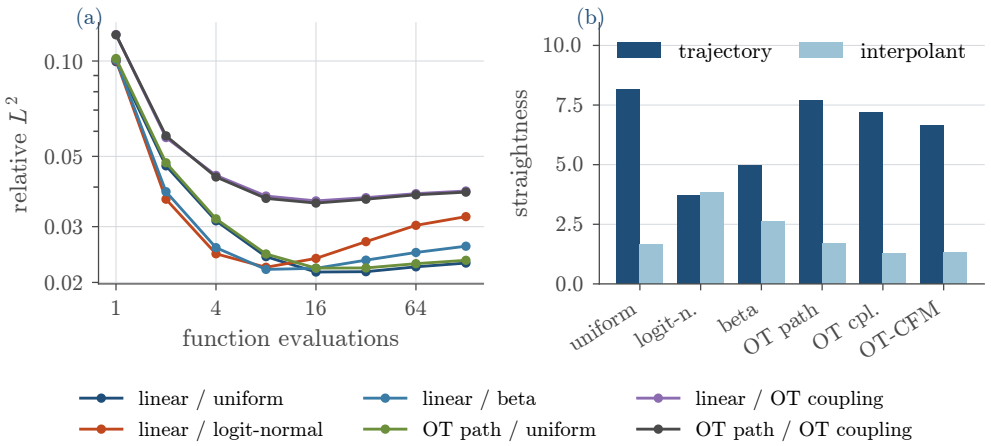

Fig. 8. **(a)** Error against function evaluations, Euler, averaged over three seeds. The curves cross: the configuration that wins below eight evaluations loses above sixteen. **(b)** The two straightness definitions disagree about which configuration is straight, and only the trajectory one predicts (a).

Against the solver it replaces. Both sides were timed on the same device (CPU, eight threads), batched over the same 200 conditions. The reference conjugate-gradient solve costs 9.7 ms per solution. The ConvNet surrogate costs 18.7 ms at one evaluation and 977 ms at fifty.

So the surrogate is *never* cheaper: 1.9× the solver at its cheapest setting, and 101× at the setting the accuracy tables use. On a linear elliptic problem at 64×64, against a well-implemented batched conjugate gradient, a neural surrogate of this size does not pay for itself, and the forward experiments are not a speed argument. The case for the method is the one Section 4.6 makes, where the classical alternative is not one solve but thousands *per observation*.

4.4 Inverse problems and calibration

E5 moves to the setting the generative argument was made for. The observation model is (16) with $\mathscr{O}$ a random restriction keeping 15% of the grid and $\sigma = 2 \times 10^{-4}$, and the target is either κ alone, conditioning on (u_{obs}, f, m), or the pair (κ, f) jointly.

First we need a reference level. A null-conditioned model here is a generative *prior* over κ: it produces admissible fields that have nothing to do with the observation, and it scores single-draw relative L^2 of 1.069 and 1.104. That is what "no information was used" looks like on this benchmark, so every comparison below keeps one of these in the table.

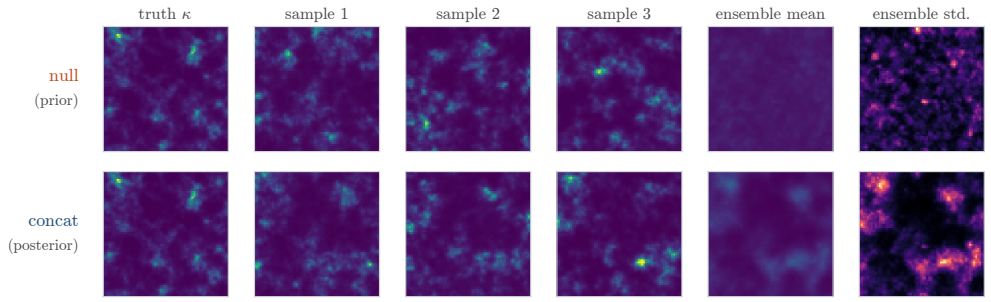

Fig. 9. Posterior draws for one test condition, joint (κ, f) target at 5000 samples, with the κ channel shown. The unconditional model produces admissible coefficient fields unrelated to the observation, and its ensemble mean is featureless. The conditioned model places structure where the truth has it, and its ensemble standard deviation is spatially organised rather than uniform, which is the quantity Figure 10 measures. The joint target is shown because the κ-only null run has no checkpoint.

A single draw hides half of what conditioning buys. A single draw from a correct posterior *should* differ from the truth by roughly the posterior width. So R3 draws $K = 32$ samples per condition and scores them properly, in Table 3.

In both matched null-versus-concat pairs the proper scores show a larger effect than the point metric does. On the κ target a single draw says conditioning buys 12.8% while CRPS says 18.6%. On the joint target the single draw says 13.8% and CRPS says 25.4%.

Which tells us that the conditioned models are not only closer to the truth. They also place a *better-shaped* distribution on it, and judged on relative L^2 alone, half of what conditioning bought is invisible.

Target	Cond.	CRPS	ES	cov. 90	spr/skl	$\rho_{e,s}$	L^2 (1)
Inverse, 5000 samples, 100 epochs							
$\kappa^{\ddagger}$	null	0.860	95.9	0.826	0.90	−0.89	1.069
$\kappa^{\ddagger}$	concat	**0.700**	**83.8**	0.838	0.92	−0.36	0.932
(κ, f)	null	0.631	100.2	0.825	0.92	−0.05	1.104
(κ, f)	concat	**0.471**	**84.8**	0.847	0.94	+0.02	0.952
Inverse, 5000 samples, 500 epochs							
κ	concat	0.747	94.0	0.703	0.47	+0.38	0.794
Forward, for contrast: true posterior is a Dirac							
u (ConvNet)	concat	$3.5{\cdot}10^{-5}$	0.003	0.959	1.16	+0.88	0.075
u (UNet)	concat	$1.5{\cdot}10^{-4}$	0.012	0.651	0.22	+0.77	0.136

Table 3. Distributional scores, $K = 32$ at a fixed seed, in physical units. ES is the energy score. "spr/skl" is the spread–skill ratio after the $\sqrt{(K+1)/K}$ correction, where 1 means calibrated and below 1 means over-confident. $\rho_{e,s}$ is the rank correlation between predicted spread and actual error, and L^2 (1) is a *single* draw's relative L^2. Rows marked ‡ are the reference cell of E9, scored on all 200 test conditions at ensemble seed 123, and the rest are the R3 re-evaluation on 64 conditions at ensemble seed 7. Comparisons are made within a protocol.

The posterior width follows the flux. Section 2.2.2 argued that κ enters only through the flux, so the observation constrains it where ∇u is large and barely at all where the solution is flat. Does the learned posterior know this? We bin every pixel of every test condition by local $|\nabla u|$ and look.

The median posterior standard deviation falls from 0.907 in the lowest flux decile to 0.234 in the highest, a factor of 3.9, with rank correlation −0.53 over 260,000 pixels. The model was never told any of this.

We consider this our best result. The model does not just return an error bar next to a point estimate. It returns a posterior whose spatial structure matches what the physics says about where κ can be recovered.

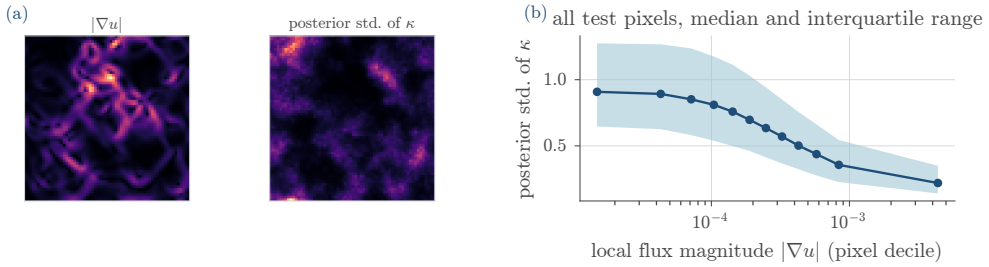

Fig. 10. Spatially varying identifiability. **(a)** $|\nabla u|$ and the posterior standard deviation of κ for one condition; the bright regions coincide. **(b)** All test pixels by flux decile: posterior width falls 3.9× from lowest to highest.

On the inverse problem the backbone stops mattering. E6 repeats the E3 backbone comparison in the inverse direction. All four networks land between 0.788 and 0.820, so the worst exceeds the best by 4%, whereas on the forward problem the same four span a factor of 2.3.

The reason is that once the observation determines the target only up to a wide posterior, the quality of the velocity field is no longer what limits the result. The information in the data is. We reach the same conclusion from the other side in Section 4.3, where a flat error-against-budget curve tells us the integrator was not the limit either.

Calibration: one result and one warning. Run the same suite on two *forward* models, whose true posterior is a point mass and whose correct spread is zero, and we get spread–skill ratios of 1.16 and 0.22. Two models approximating the same posterior report uncertainties differing by a factor of five, which makes Remark 5 quantitative. In practice this means we should always ensemble on a forward problem, since averaging 32 draws cuts the ConvNet's relative L^2 from 0.075 to 0.035, and we should never call the result uncertainty quantification.

The warning is the κ cell trained for 500 epochs instead of 100. Its single-draw error improves from 0.932 to 0.794, while its spread–skill ratio falls from 0.92 to 0.47 and its 90% coverage from 0.84 to 0.70, with

the U-shaped rank histogram of under-dispersion in Figure 11(a). More optimization made the posterior narrower than the evidence supports. A point metric rewards this and a proper scoring rule penalizes it, which is why we run both.

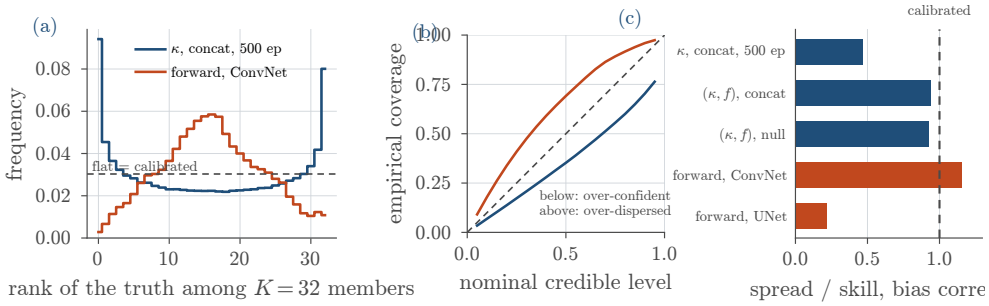

Fig. 11. Calibration, $K = 32$. **(a)** Rank histograms: the inverse model is U-shaped (under-dispersed) and the forward one dome-shaped. **(b)** Reliability against the diagonal. **(c)** Bias-corrected spread–skill; the two forward models, whose true posterior is identical and degenerate, disagree by a factor of five.

4.5 Controlling how ill-posed the problem is

Everything above observes 15% of the grid at one noise level, which makes the degree of posterior degeneracy an accident of a single setting. It need not be. The mask fraction and the observation noise are generator parameters, so the amount of information in the observation is something we can vary.

E9 varies it on Darcy, with eight cells trained identically and each scored with the full $K = 32$ suite. E11 and E12 then do the same on the two benchmarks that until now appeared only in the forward direction, which answers a question the Darcy sweep cannot: whether the behaviour belongs to the method or to that one problem.

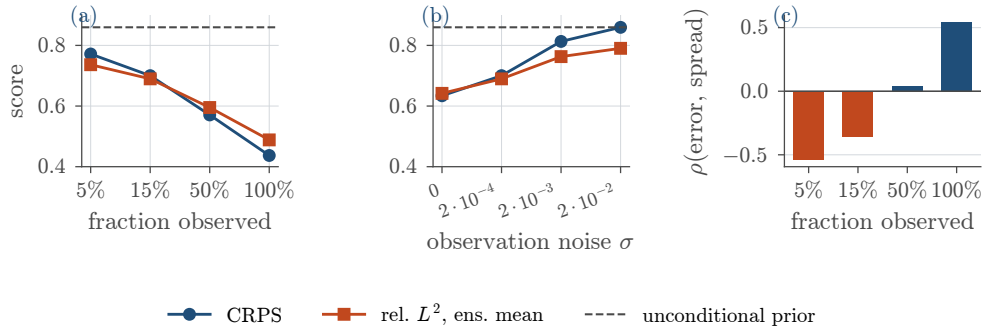

Fig. 12. Inverse Darcy, κ target, as the observation is degraded. **(a)** Observed fraction at $\sigma = 2 \times 10^{-4}$. **(b)** Noise at 15% observed. In both, the dashed line is the unconditional model, so a curve reaching it is a model that has stopped using its data. **(c)** Rank correlation between per-sample ensemble spread and per-sample error, which changes sign across the sweep.

Both ends of the sweep are anchored. Going from 5% to full observation moves CRPS from 0.772 to 0.437, and ensemble-mean relative L^2 from 0.736 to 0.488. Going from noiseless to $\sigma = 2 \times 10^{-2}$ moves CRPS from 0.634 to 0.860.

That last number is worth comparing against the unconditional prior, which also scores 0.860, to three decimal places. So at a hundred times the reference noise the conditional model has ended up *exactly* at the prior. It still produces admissible κ fields, but they carry no information about the observation.

This means the upper end of the range is not an arbitrary bad number. It is a measured floor, and it shows where a model actually lands once the data are destroyed.

The same sweep on two other PDEs. Table 4 repeats it on Poisson ($u \to f$) and Burgers ($u(T) \to u(0)$), fully observed so that noise is the only degradation, with the two ladders matched in units of the observation's own standard deviation.

Every column is monotone with no crossings. The clean Poisson inverse is nearly exact, at CRPS 0.0072 and ensemble-mean relative L^2 0.014, which tells us the inverse direction is not hard in itself. It is hard exactly to the extent that the observation is uninformative.

The two problems also degrade at very different rates, and Figure 3 predicts which one degrades faster. Thirty per cent of relative noise multiplies Poisson's CRPS by 17.5 and Burgers' by 1.7, because inverting the Poisson operator amplifies exactly the band the noise occupies.

	σ	σ/std	rel. L^2		CRPS	cov. 90%	spread/skill
			1 draw	mean			
Poisson	0	0.00	0.047	0.014	0.0072	0.998	2.96
	0.0452	0.30	0.476	0.359	0.1267	0.815	0.91
	0.0905	0.60	0.775	0.572	0.2037	0.823	0.93
	0.1357	0.90	0.960	0.700	0.2519	0.830	0.95
Burgers	0	0.00	0.469	0.297	0.1257	0.879	0.89
	0.0769	0.30	0.810	0.553	0.2121	0.836	0.89
	0.1538	0.60	0.984	0.651	0.2451	0.828	0.90
	0.2307	0.90	1.102	0.723	0.2707	0.822	0.90

Table 4. Noisy inversion on Poisson ($u \to f$) and Burgers ($u(T) \to u(0)$). "1 draw" is a single posterior sample and "mean" the $K = 32$ ensemble mean, both in physical units on the test split. Coverage is against a nominal 0.90. One run per cell at seed 42.

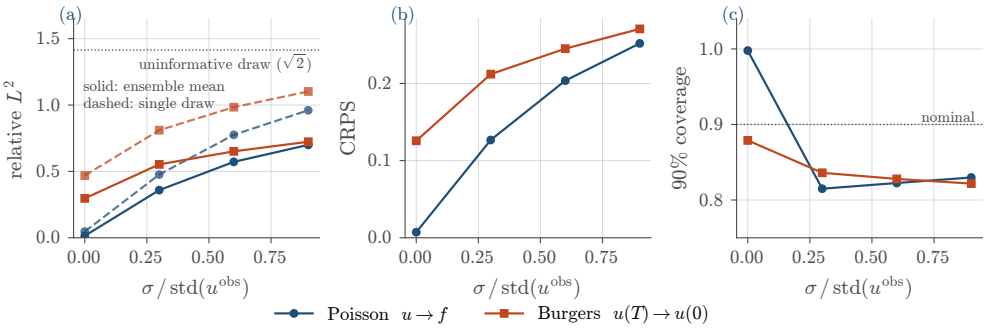

Fig. 13. Noise sweeps on both inverse problems, against noise measured in units of the observation's own standard deviation so that the two benchmarks share an axis. **(a)** Relative L^2: the ensemble mean (solid) beats a single draw (dashed) by 1.35–2.3× everywhere. **(b)** CRPS. **(c)** 90% coverage: the clean Poisson cell is the outlier, and it is the one whose posterior is nearly a Dirac.

Calibration survives the degradation. Across all eight Darcy cells the corrected spread–skill ratio stays between 0.88 and 0.93, and 90% coverage between 0.83 and 0.85, while CRPS nearly doubles. The Poisson and Burgers ladders sit in the same band.

So the models track how much they know even as what they know collapses. This is the property we wanted from the method, and it is worth keeping apart from accuracy: a model can be uninformative and still honest about it, and these ones are.

The exception, in both directions, is a near-Dirac posterior. The clean Poisson cell at spread–skill 2.96 is the one *over*-dispersed model in the report, for the same reason the forward models of Section 4.4 are unreliable there.

One caveat applies to the whole subsection. Spread is a usable error bar only where the problem is well determined. The rank correlation between per-sample spread and per-sample error runs −0.54 at 5% observed, −0.36 at 15%, +0.04 at 50% and +0.54 at full observation, so it changes sign.

The reason is the following. Where the observation is uninformative the model falls back towards the prior, and prior draws are both wide and, for a field near the prior mean, not especially wrong. So the widest predictions are not the worst ones. Sampler spread only becomes a usable error bar once the data determine the answer, which is unfortunately when we need it least.

4.6 Comparison against MCMC

Everything above shows that the inverse samplers are calibrated and that they use their data. Whether they approximate the *Bayesian* posterior is a different question, and answering it needs a reference. Getting one is the expensive part.

The natural algorithm is preconditioned Crank–Nicolson [5], and it is natural for a reason specific to this generator. Here κ is built by pushing white noise $\boldsymbol{\xi} \sim N(\mathbf{0}, \boldsymbol{I})$ through a fixed spectral filter (18),

so sampling in $\boldsymbol{\xi}$ rather than in κ makes the prior exactly Gaussian. It then cancels from the acceptance ratio, and the proposal

$$\boldsymbol{\xi}' = \sqrt{1-\beta^2}\,\boldsymbol{\xi} + \beta\boldsymbol{\zeta}$$

preserves it exactly. So the acceptance rate does not degenerate with grid refinement, which is what makes a 4096-dimensional chain tractable at all.

On Darcy the reference could not be obtained. E10 ran four test observations, two chains each, 6000 burn-in and 20,000 sampling iterations, with one forward solve per chain per iteration. The sampler behaved as designed and acceptance settled at 0.20. But 52,000 forward solves per observation were not enough. Split-$\hat{R}$ runs from 1.05 to 1.89, and the effective sample size from 20,000 draws is between 12 and 39.

The clearest sign is this. Two chains targeting the *same* posterior disagree on the posterior mean by 0.38 to 0.99 relative L^2, while two chains targeting *different* observations disagree by 1.04. We therefore draw no posterior comparison from this run, since a reference that disagrees with itself cannot be used as one.

What the failure does give us is a cost. Seven hundred and one seconds and 52,000 PDE solves per observation bought nothing usable, while the trained flow produced its $K = 32$ ensemble in 16.1 seconds and *zero* forward solves.

On a smaller problem the chain converges. E13 finds the regime where it does. On Burgers at 256 points, recovering $u(0)$ from a noisy $u(T)$, two settings sit either side of the identifiability frontier. Four initial-condition modes at $T = 0.1$ is where pCN converges, at $\hat{R} \leq 1.013$ with effective sample sizes of 578 to 1581. Eight modes at $T = 0.4$ is where it does not, at $\hat{R}$ between 5.2 and 6.8.

Against the converged reference the result is not in our favour, and we report it anyway. pCN recovers $u(0)$ to 0.004–0.010 relative L^2, where the flow reaches 0.030–0.061, so a factor of five to ten, and the flow's posterior is 6.6–11.1× too wide. Where the data nearly determine the answer, the learned sampler does not become sharp enough. Note that this is the opposite of the over-confidence Section 4.4 found on the hard problems.

Past the frontier the picture inverts. There the two widths agree to within 12–39%, so the flow's spread is about right exactly where the posterior is genuinely broad.

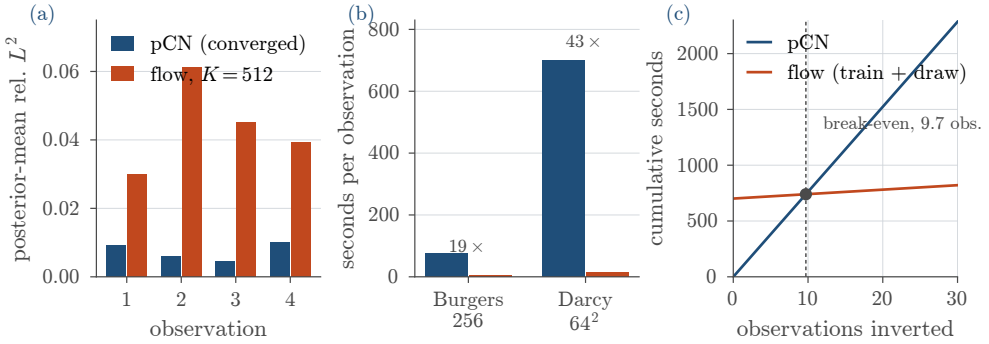

Fig. 14. E13, Burgers 256, identifiable regime, against E10's Darcy reference. **(a)** Posterior-mean error where the chain converged: pCN is five to ten times more accurate. **(b)** Cost per observation on the same machine. **(c)** Cumulative cost against observations inverted. The surrogate's training is a fixed offline cost and its per-observation cost is nearly flat, so the two lines cross at ten observations.

The break-even is ten observations. This is the number the reduced-order argument rests on, and it is small. Training cost 701 seconds, once. Each new observation costs the chain 76 seconds and the flow 4.0, so the cumulative curves cross at 9.7 observations, and that figure is stable to within 1% across all four test observations.

Which gives a simple rule. Below ten inversions, run the chain. Above ten, the surrogate has paid for itself and every further observation is nearly free. That is the offline/online split of Section 2.2 as a measurement rather than as an argument.

So what we can claim is not that the learned posterior is better. On the one problem where we can check it, it is worse. What we can claim is that it is *approximately right and amortized*, and that we have now measured what that trade costs.

What this does not establish. That vanilla pCN at this budget fails on Darcy does not mean Markov chain Monte Carlo is infeasible there. pCN takes the same step size in all 4096 directions, while the observation constrains only a few hundred of them. A dimension-reduced or gradient-informed sampler would very likely mix at a budget this one did not, and it would also be the right instrument for checking whether the 6.6–11.1× over-dispersion measured on Burgers persists at 64×64.

We should also record one withdrawn measurement. A companion forward-cost sweep on the same Burgers problem was run and then discarded. At $\nu = 10^{-2}$ the solution has decayed to numerical zero by $T = 6.4$, so the apparent 6.4× surrogate advantage at the longest horizon was an artefact of the solver integrating 25,600 steps to produce nothing. At the one non-degenerate horizon the solver wins outright, which agrees with Section 4.3.

4.7 Threats to validity

Replication is partial. E2, E4 and E7 were rerun at three matched seeds, and in both places where a seed variance could be measured it was small against the effect, at 0.5–1.8% on the Burgers backbones and 1.6–5.0% on the components. The exception is maximum-likelihood training at 39.3%. E3, E5, E6 and E8–E13 remain single runs at a seed that differs between variants, so strictly speaking Section 4.2's Darcy panel compares ConvNet-at-seed-42 with UNet-at-seed-2042. The Burgers replication makes that a mild concern rather than an open one, and the Darcy replication was started and abandoned.

No deterministic baseline. Nothing here is compared against a regressor trained on the same data. The Fourier neural operator reaches roughly one per cent on the standard benchmarks [19], though the Darcy variant we use is the harder parametric one and is not directly comparable to that figure.

The learned posterior is checked on one problem only. E13 gives a converged reference on Burgers at 256 points, and finds the flow 5–10× less accurate and 6.6–11.1× over-dispersed there. Whether either number transfers to Darcy at 64×64 is unknown, because E10 did not obtain a reference. This is our largest gap, and closing it needs a better sampler rather than more iterations of the same one.

One dataset size on the forward problems. Every forward experiment uses 3000 samples, and Section 4.4 shows that the training budget alone can decide a conclusion. The inverse side varies observation quality and, across experiments, budget and data size. No forward experiment varies either.

Backbones matched by configuration, not by budget. They differ by an order of magnitude in parameters and by a factor of two and a half in cost per evaluation, and in opposite directions. On Darcy the ConvNet is both more accurate *and* more expensive, at 0.43 s per training step against 0.17 s. So "smaller is better here" is a statement about parameters and not about compute.

Two conclusions rest on a single problem. E7 ran on Burgers 1D because the whole matrix is affordable there, so whether logit-normal sampling buys the same few-step accuracy on Darcy, where trajectories are far more curved, is untested. E4's objective comparison is one budget on one 16×16 problem, and the variants that would have answered whether more probes close the gap did not complete.

Measurement details. The original runs averaged metrics per batch and the re-evaluations per sample, and the two agree within a few per cent, with E3's ConvNet at 0.0734 recorded and 0.0701 recomputed. The R3 and E9 uncertainty suites use different numbers of test conditions and different ensemble seeds, so Table 3 marks which protocol produced each row. Finally, E5's runner was not committed alongside its run and had to be reconstructed from the resolved configurations, and one of its cells, κ-only null at 5000, saved no checkpoint. That is why the corresponding baseline in Table 3 is taken from E9's reference cell instead.

5 Conclusions

We built a library in which the parts of a flow-based PDE surrogate are separate components, and we used it to compare those parts on Poisson, Burgers and Darcy problems.

The main results are the following.

Conditioning matters, and the training loss does not show it. Removing the conditioner costs a factor of 17 in median relative L^2 while both loss curves still look healthy, so we select models on sampled error instead (Section 4.2).

The smallest of four backbones is the most accurate on both forward problems, by 12.4× on Burgers, and attention adds nothing beyond seed noise (Section 4.2). At this problem size, a bigger network is not what improves the result.

The sampler is a real design axis. Solver and step count are chosen after training, on fixed weights, and there first-order Euler beats fourth-order Runge–Kutta at equal cost by up to 8.6×. Integrating the ODE all the way to convergence is also not the best choice, since sixteen Euler steps give 11% lower error than the adaptive solver on our best model (Section 4.3).

On the inverse problems, the choice of metric changed the conclusion. Proper scoring rules register about twice the effect of conditioning that a single draw's relative L^2 does. The same scores also caught a model that looked better after longer training while becoming badly over-confident, with 90% coverage falling from 0.84 to 0.70 (Section 4.4).

The learned posterior carries structure we never put into it. Binned by local flux magnitude, its width falls by 3.9× from the flattest decile to the steepest one, which is where Darcy theory says κ is identifiable. We consider this our best result.

Finally, on cost. Against a classical pCN chain the learned sampler is five to ten times less accurate on the one problem where the chain converged. But training is paid once, and each further observation costs the chain 76 seconds against the flow's 4.0, so the two break even after ten inversions (Section 4.6).

Limitations. Most cells are a single run, and the seed differs between variants, so only E2, E4 and E7 are properly replicated. We have no deterministic baseline trained on the same data. Every forward experiment uses one dataset size. And the learned posterior is checked against a converged reference on one problem only, because on Darcy the reference chain did not mix. Section 4.7 lists these in full.

References

[1] M. S. Albergo and E. Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023.

[2] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. `http://github.com/jax-ml/jax`.

[3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[4] H. Chung, J. Kim, M. T. Mccann, M. L. Klasky, and J. C. Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations (ICLR)*, 2023.

[5] S. L. Cotter, G. O. Roberts, A. M. Stuart, and D. White. MCMC methods for functions: Modifying old algorithms to make them faster. *Statistical Science*, 28(3):424–446, 2013. doi: 10.1214/13-STS421.

[6] J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[7] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning (ICML)*, 2024.

[8] V. Fortin, M. Abaza, F. Anctil, and R. Turcotte. Why should ensemble spread match the RMSE of the ensemble mean? *Journal of Hydrometeorology*, 15(4):1708–1713, 2014.

[9] S. Fresca and A. Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.

[10] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[11] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. FFJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] T. M. Hamill. Interpretation of rank histograms for verifying ensemble forecasts. *Monthly Weather Review*, 129(3): 550–560, 2001.

[13] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[14] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[15] M. F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics — Simulation and Computation*, 18 (3):1059–1076, 1989.

[16] F. Koehler, S. Niedermayr, R. Westermann, and N. Thuerey. APEBench: A benchmark for autoregressive neural emulators of PDEs. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[17] G. Kohl, K. Um, and N. Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. In *International Conference on Machine Learning (ICML)*, 2024.

[18] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89): 1–97, 2023.

[19] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[20] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[21] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat. Flow matching guide and code. *arXiv preprint arXiv:2412.06264*, 2024.

[22] P. Lippe, B. S. Veeling, P. Perdikaris, R. E. Turner, and J. Brandstetter. PDE-Refiner: Achieving accurate long rollouts with neural PDE solvers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[23] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[25] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637:84–90, 2025.

[26] A. Quarteroni, A. Manzoni, and F. Negri. *Reduced Basis Methods for Partial Differential Equations: An Introduction*, volume 92 of *UNITEXT*. Springer, 2016.

[27] D. J. Rezende and S. Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning (ICML)*, pages 1530–1538, 2015.

[28] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[29] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[30] A. M. Stuart. Inverse problems: A Bayesian perspective. *Acta Numerica*, 19:451–559, 2010.

[31] G. J. Székely and M. L. Rizzo. Energy statistics: A class of statistics based on distances. *Journal of Statistical Planning and Inference*, 143(8):1249–1272, 2013.

[32] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024.

[33] J. Wildberger, M. Dax, S. Buchholz, S. R. Green, J. H. Macke, and B. Schölkopf. Flow matching for scalable simulation-based inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.